ASSIGNMENT 3 – CO3

Python Data Structures: Lists, Tuples, Dictionaries, Sets and Comprehensions

SUBMITTED BY:- M SANTHOSH KUMAR(192412268)

Q1 – Lists and Slicing: Daily Rainfall Records (6 Marks)

Implementation

```
import random
import time

def weekly_totals(readings):
    """Return total rainfall for each consecutive 7-day block."""
    return [sum(readings[i:i+7]) for i in range(0, len(readings), 7)]

def moving_average_naive(readings, window=7):
    """Return averages for complete windows, using a fresh sum each time."""
    if window <= 0 or len(readings) < window:
        return []
    return [sum(readings[i:i+window]) / window
            for i in range(len(readings) - window + 1)]

def moving_average_running(readings, window=7):
    """Return averages for complete windows using a running sum."""
    if window <= 0 or len(readings) < window:
        return []
    current = sum(readings[:window])
    result = [current / window]
    for i in range(window, len(readings)):
        current += readings[i] - readings[i-window]
        result.append(current / window)
    return result

def dry_days(readings):
    """Return zero-based day indices with zero rainfall."""
    return [i + 1 for i, value in enumerate(readings) if value == 0]

if __name__ == "__main__":
    random.seed(42)
    readings = [random.randint(0, 100) for _ in range(365)]
    weekly = weekly_totals(readings)
    print("Weekly blocks:", len(weekly), "Wettest block:", max(weekly))
    print("Driest block:", min(weekly), "Dry days:", dry_days(readings))
    print("7-day averages:", moving_average_running(readings)[:5])
    print("readings[7:14] =", readings[7:14])
    print("readings[-7:] =", readings[-7:])
    print("readings[::7] =", readings[::7])
```

Pseudocode for a running-sum moving average:

```
IF number of readings < 7, return empty list
SET window_sum = sum of first 7 readings
ADD window_sum / 7 to result
FOR each next reading at index i from 7 to end
    window_sum = window_sum + readings[i] - readings[i-7]
    ADD window_sum / 7 to result
RETURN result
```

readings[7:14] returns items at indices 7 through 13 (seven values). readings[-7:] returns the final seven values. readings[::7] returns every seventh value, beginning at index 0. A list suits daily ordered records and supports slicing and append. append is amortized O(1); insert at an arbitrary position is O(n) due to shifting; pop at the end is O(1) amortized, while pop from the front is O(n); sort is O(n log n) in the general case.

For m = 7, the naive method sums 7 values for each of n−6 windows, about 7(n−6) additions. The running-sum method computes the first 7-value sum, then performs one addition and one subtraction for each later window, giving O(n) work rather than O(7n), which is also O(n) for fixed window size. Empty inputs and inputs shorter than 7 return an empty moving-average list.

Timing investigation

```
def benchmark_rainfall():
    random.seed(7)
```

```
    for n in (365, 36_500, 365_000):
        data = [random.randint(0, 100) for _ in range(n)]
        start = time.perf_counter()
        naive = moving_average_naive(data)
        naive_time = time.perf_counter() - start
        start = time.perf_counter()
        fast = moving_average_running(data)
        fast_time = time.perf_counter() - start
        assert all(abs(a-b) < 1e-9 for a, b in zip(naive, fast))
        print(n, len(naive), f"naive={naive_time:.6f}s",
              f"running={fast_time:.6f}s")

benchmark_rainfall()
```

A late reading inserted into a Python list at the correct date costs O(n) because later elements shift. For a few hundred annual readings, a list remains practical. For frequent insertions or large historical data, a date-keyed dictionary or database may be better. A deque(maxlen=7) supports efficient end operations and a rolling window; a date dictionary offers date lookup but needs sorting or a separate ordered index for chronological reporting. A naive 7-day window is O(7n), while a running sum is O(n). The list uses O(n) space; a streaming deque can use O(7) window space when the full history is not required.

Q2 – Tuples, Mutability and Copying (7 Marks)

```
import copy
import sys
import time

rows, cols = 3, 4
bad = [[0] * cols] * rows
bad[0][1] = 7
good = [[0 for _ in range(cols)] for _ in range(rows)]
good[0][1] = 7
print("bad :", bad)
print("good:", good)
print(bad[0] is bad[1], good[0] is good[1])

stops = {
    (13.0827, 80.2707): "Chennai Central",
    (13.0418, 80.2341): "T. Nagar"
}
print(stops[(13.0418, 80.2341)])

# Aliasing and copies
original = [[1, 2], [3, 4]]
alias = original
shallow_copy = copy.copy(original)
slice_copy = original[:]
deep_copy = copy.deepcopy(original)
original[0][0] = 99
print("alias:", alias)
print("shallow:", shallow_copy)
print("slice:", slice_copy)
print("deep:", deep_copy)

# Tuple may contain a mutable list
mixed = ("bus stop", [10, 20])
mixed[1].append(30)
print(mixed)

print("list size:", sys.getsizeof(list(range(1_000_000))))
print("tuple size:", sys.getsizeof(tuple(range(1_000_000))))
```

Expected grid result: bad rows all change at column 1 because each row refers to the same inner list; good rows are independent. The correct grid is created with [[0 for _ in range(cols)] for _ in range(rows)]. Tuples can be dictionary keys when all their elements are hashable; a list is mutable and unhashable, so using [13.0418, 80.2341] as a key raises TypeError.

Memory diagram: bad = [row, row, row], where each slot points to the same inner list. good = [row1, row2, row3], where each row is a distinct list. An alias is another reference to the same object. A shallow copy creates a new outer container but shares

nested objects; a deep copy recursively copies nested objects. A shallow copy is sufficient when contained elements are immutable or shared nested state is intentional. Use deep copy when independent nested structures are required.

A function that mutates a list argument changes the caller-visible object because both names refer to the same list. A tuple is immutable only at the outer level; if it contains a list, that list can still be mutated. Tuple immutability protects the tuple's element bindings from reassignment and makes hashable tuples usable as dictionary keys.

Tuple/list experiment

```
values_list = list(range(100_000))
values_tuple = tuple(values_list)

start = time.perf_counter()
_ = values_list[50_000]
list_index_time = time.perf_counter() - start

start = time.perf_counter()
_ = values_tuple[50_000]
tuple_index_time = time.perf_counter() - start

print("indexing:", list_index_time, tuple_index_time)
print("identity:", bad[0] is bad[1], good[0] is good[1])
print("ids:", id(bad[0]), id(bad[1]), id(good[0]), id(good[1]))
```

sys.getsizeof() measures the container object itself, not necessarily all nested objects. The assignment asks for one million values, so run the provided memory lines and report the actual byte counts for your Python version. To compare creation fairly, time list(range(n)) and tuple(range(n)) separately for the same n, using multiple repetitions.

Sparse zone grid alternative

```
# Store only zones with passenger counts greater than zero
zone_counts = {}
zone_counts[(1, 2)] = 5
zone_counts[(2, 3)] = 2
print(zone_counts.get((0, 0), 0))  # empty zone returns zero
```

List-of-lists access and dictionary access are both O(1) average for a known cell/key. A dense grid uses O(rows × cols) space; a sparse dictionary uses O(number of non-empty zones) space, but has additional hash-table overhead.

Q3 – Dictionaries: Farmer Helpline Complaint Analysis (7 Marks)

```
import random
import string
import time
from collections import Counter

calls = [
    "seeds not received seeds delayed",
    "fertiliser price high",
    "seeds price high subsidy not received",
]
STOP = {"not", "high"}

def frequencies(texts, stop):
    """Count case-normalized words, excluding stop words and punctuation."""
    counts = {}
    for text in texts:
        cleaned = text.lower().translate(str.maketrans("", "", string.punctuation))
        for word in cleaned.split():
            if word not in stop:
                counts[word] = counts.get(word, 0) + 1
    return counts

def top(counts, k):
    """Return top k (word, count) pairs by count descending, then word."""
    return sorted(counts.items(), key=lambda item: (-item[1], item[0]))[:k]
```

```
print(top(frequencies(calls, STOP), 3))

# Group by district and crop
complaints = [
    {"district": "A", "crop": "rice", "text": "seeds delayed"},
    {"district": "B", "crop": "cotton", "text": "price high"},
    {"district": "A", "crop": "rice", "text": "fertiliser missing"},
]
by_district = {}
by_crop = {}
for item in complaints:
    by_district.setdefault(item["district"], []).append(item["text"])
    by_crop.setdefault(item["crop"], []).append(item["text"])
print(by_district)
print(by_crop)
```

Expected top output: [('seeds', 3), ('price', 2), ('received', 2)]. price comes before received because the sorting rule uses descending count and then alphabetical order for ties.

A dictionary uses a hash of the key to select a table location; average lookup is O(1) under typical hash-distribution assumptions, although worst-case collisions can be slower. Counting with a dictionary or Counter takes O(n) expected time for n word tokens. Calling list.count() for every distinct word takes O(n·d), where d is the number of distinct words. Python 3.7+ guarantees dictionary insertion order. keys(), values() and items() return dynamic view objects reflecting the dictionary's current contents.

Performance experiment

```
def benchmark_word_count():
    vocabulary = ["seed", "rice", "price", "water", "farmer", "crop"]
    for n in (1_000, 10_000, 100_000):
        generated = [f"{random.choice(vocabulary)} {random.choice(vocabulary)}"
                     for _ in range(n)]
        tokens = " ".join(generated).split()
        start = time.perf_counter()
        d = {}
        for w in tokens:
            d[w] = d.get(w, 0) + 1
        dict_time = time.perf_counter() - start

        start = time.perf_counter()
        c = Counter(tokens)
        counter_time = time.perf_counter() - start

        # List.count for each distinct token is intentionally slower.
        start = time.perf_counter()
        distinct = set(tokens)
        slow = {w: tokens.count(w) for w in distinct}
        list_count_time = time.perf_counter() - start
        assert d == c == slow
        print(n, f"dict={dict_time:.6f}s",
              f"Counter={counter_time:.6f}s",
              f"list.count={list_count_time:.6f}s")

benchmark_word_count()
```

Test cases should include empty text (no tokens), mixed uppercase/lowercase (normalized by lower()), and punctuation (removed before splitting). For 100,000 complaints per day, use a dictionary or Counter because expected time grows linearly with the number of tokens. Counter is concise and optimized; an explicit dictionary loop is useful when custom filtering and processing are required. The count dictionary uses O(d) space.

Q4 – Sets: Elective Registration Clash Detection (7 Marks)

```
ai_elective = {"192311001", "192311004", "192311007", "192311009"}
iot_elective = {"192311002", "192311004", "192311009", "192311010"}
all_students = {f"1923110{i:02d}" for i in range(1, 11)}

both = ai_elective & iot_elective
```

```
exactly_one = ai_elective ^ iot_elective
neither = all_students - (ai_elective | iot_elective)

print("clash      :", sorted(both))
print("exactly one:", sorted(exactly_one))
print("neither    :", sorted(neither))

# Equivalent set methods
both2 = ai_elective.intersection(iot_elective)
exactly_one2 = ai_elective.symmetric_difference(iot_elective)
neither2 = all_students.difference(ai_elective.union(iot_elective))
assert (both, exactly_one, neither) == (both2, exactly_one2, neither2)
```

A Venn diagram can be described as two overlapping circles: the intersection is the overlap; symmetric difference is the two non-overlapping portions; neither is the area of the universal set outside both circles. Sets remove duplicates automatically and are unordered, so sorted() is used for stable reports. Membership is average O(1) for sets versus O(n) for lists. Lists are unhashable and cannot be set elements; tuples of hashable items can be set elements.

Converting a list to a set is useful when duplicate values do not matter and fast membership/intersection is needed. It loses duplicate counts and original order.

Membership and intersection benchmark

```
def benchmark_sets():
    for n in (10_000, 100_000, 1_000_000):
        values = [str(i) for i in range(n)]
        value_set = set(values)
        target = str(n - 1)
        start = time.perf_counter()
        found_list = target in values
        list_time = time.perf_counter() - start
        start = time.perf_counter()
        found_set = target in value_set
        set_time = time.perf_counter() - start
        assert found_list == found_set
        print(n, f"list={list_time:.8f}s", f"set={set_time:.8f}s")

benchmark_sets()
```

For two lists of sizes n and m, nested-loop intersection takes O(n·m). Sorting both lists then walking through them takes O(n log n + m log m) time. Set intersection is O(n + m) on average and uses O(n + m) space to hold the sets/results. Test the empty-set case and identical electives; identical sets have an empty symmetric difference and their intersection equals the original set.

Q5 – List Comprehension: Cleaning Sensor Readings (6 Marks)

```
import random
import time
import sys

readings = [2.0, 3.5, 52.0, -60.0, 8.5, 1.0, 0.0, 51.0]

# 1. for loop with append
valid_loop = []
for c in readings:
    if -50 <= c <= 50:
        valid_loop.append(c)
fahrenheit_loop = [(c * 9/5) + 32 for c in valid_loop]

# 2. list comprehension
valid_comp = [c for c in readings if -50 <= c <= 50]
fahrenheit_comp = [(c * 9/5) + 32 for c in valid_comp]

# 3. filter/map and lambda
valid_map = list(filter(lambda c: -50 <= c <= 50, readings))
fahrenheit_map = list(map(lambda c: c * 9/5 + 32, valid_map))

alarm_minutes = [
```

```
    minute for minute, c in enumerate(readings, start=1)
    if -50 <= c <= 50 and not (2 <= c <= 8)
]

assert valid_loop == valid_comp == valid_map
assert fahrenheit_loop == fahrenheit_comp == fahrenheit_map
print("valid C:", valid_comp)
print("valid F:", fahrenheit_comp)
print("alarm minutes:", alarm_minutes)

# Dictionary and set comprehensions
fahrenheit_by_minute = {
    minute: c * 9/5 + 32 for minute, c in enumerate(readings, start=1)
    if -50 <= c <= 50
}
alarm_values = {c for c in readings if -50 <= c <= 50 and not (2 <= c <= 8)}
print(fahrenheit_by_minute)
print(alarm_values)
```

General form: [expression for item in iterable if condition]. A comprehension is concise for a simple transform/filter; a normal loop is easier to read when it contains several branches, error handling or side effects. A list comprehension materializes all results in O(n) memory; a generator expression yields values lazily and uses O(1) auxiliary iteration memory, excluding the source and any materialized output.

Timing and memory experiment

```
def clean_loop(data):
    result = []
    for c in data:
        if -50 <= c <= 50:
            result.append(c)
    return result

def clean_comp(data):
    return [c for c in data if -50 <= c <= 50]

def clean_filter(data):
    return list(filter(lambda c: -50 <= c <= 50, data))

for n in (10_000, 100_000, 1_000_000):
    data = [random.uniform(-60, 60) for _ in range(n)]
    outputs = []
    for fn in (clean_loop, clean_comp, clean_filter):
        start = time.perf_counter()
        out = fn(data)
        elapsed = time.perf_counter() - start
        outputs.append(out)
        print(n, fn.__name__, f"{elapsed:.6f}s")
    assert outputs[0] == outputs[1] == outputs[2]

example = list(range(100_000))
gen = (x for x in example if x % 2 == 0)
print("list object bytes:", sys.getsizeof([x for x in example if x % 2 == 0]))
print("generator object bytes:", sys.getsizeof(gen))
```

The list comprehension and generator expression should not be compared as though they store the same thing: the list stores every selected result, while the generator stores iteration state and computes values on demand. For an all-valid input, the valid output has n items; for an all-invalid input, the valid output is empty. All three cleaning implementations should produce identical lists. A comprehension is not always best; readability and whether the full result must be stored should guide the choice.

Q6 – Nested Data: A Railway Timetable (7 Marks)

```
import random
import time
import sys

timetable = {
    "12601": {
```

```
        "name": "Chennai Express",
        "stops": [
            ("Chennai", "06:00", "06:10"),
            ("Vijayawada", "12:00", "12:05"),
            ("Hyderabad", "18:00", "18:10")
        ],
        "days": {"Mon", "Wed", "Fri"}
    },
    "12602": {
        "name": "Coastal Express",
        "stops": [
            ("Chennai", "07:00", "07:15"),
            ("Nellore", "09:00", "09:05"),
            ("Vijayawada", "12:30", "12:40")
        ],
        "days": {"Tue", "Thu", "Sat"}
    },
    "12710": {
        "name": "Deccan Express",
        "stops": [
            ("Bengaluru", "05:30", "05:40"),
            ("Chennai", "10:00", "10:10"),
            ("Vijayawada", "16:00", "16:10")
        ],
        "days": {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat"}
    },
    "12840": {
        "name": "Eastern Express",
        "stops": [
            ("Hyderabad", "08:00", "08:10"),
            ("Vijayawada", "11:00", "11:05"),
            ("Visakhapatnam", "15:00", "15:10")
        ],
        "days": {"Sun", "Tue", "Thu"}
    },
    "12077": {
        "name": "Intercity",
        "stops": [
            ("Chennai", "08:30", "08:35"),
            ("Katpadi", "10:00", "10:05"),
            ("Bengaluru", "13:00", "13:10")
        ],
        "days": {"Mon", "Tue", "Wed", "Thu", "Fri", "Sat"}
    }
}

def trains_at_station(data, station, day):
    """Find trains stopping at station and running on day."""
    result = []
    for number, info in data.items():
        if day not in info["days"]:
            continue
        for name, arrival, departure in info["stops"]:
            if name.lower() == station.lower():
                result.append((departure, number, info["name"]))
                break
    return sorted(result, key=lambda row: row[0])

print("Vijayawada on Tuesday:")
for departure, number, name in trains_at_station(timetable, "Vijayawada", "Tue"):
    print(departure, number, name)

# Departure time at the third stop of a train:
# timetable["12601"]["stops"][2][2]

for number, info in timetable.items():
    print(number, info["name"], "days:", sorted(info["days"]))
    for station, arrival, departure in info["stops"]:
        print(f"  {station:15} arrival {arrival} departure {departure}")
```

The outer dictionary maps train number to train data for average O(1) lookup by train number. A nested dictionary gives named fields; a list preserves stop order; each tuple represents a fixed station/arrival/departure record; a set supports fast running-day membership. The expression for the third station's departure time is timetable['12601']['stops'][2][2].

Without an index, a station query scans trains and their stops, taking O(S) time where S is the total number of stops, plus sorting the matches. Build a station index mapping station name to a list of (train number, departure, running days). Then query cost is approximately O(T_s) to filter trains at that station, where T_s is the number of indexed trains at that station, plus sorting results. The index uses extra O(S) space and must be updated when timetable data changes.

Generate and time a larger timetable

```
    def build_index(data):
        index = {}
        for number, info in data.items():
            for station, arrival, departure in info["stops"]:
                index.setdefault(station.lower(), []).append(
                    (number, departure, info["days"], info["name"]))
        return index

    station_index = build_index(timetable)

    def query_index(index, station, day):
        rows = [
            (departure, number, name)
            for number, departure, days, name in index.get(station.lower(), [])
            if day in days
        ]
        return sorted(rows, key=lambda x: x[0])

    def make_synthetic_timetable(n):
        stations = ["Chennai", "Vijayawada", "Hyderabad", "Nellore", "Bengaluru"]
        days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
        data = {}
        for i in range(n):
            chosen = random.sample(stations, 3)
            data[f"T{i:05d}"] = {
                "name": f"Train {i}",
                "stops": [(s, "08:00", f"{(i % 24):02d}:00") for s in chosen],
                "days": set(random.sample(days, 4))
            }
        return data

    for n in (100, 1_000, 10_000):
        data = make_synthetic_timetable(n)
        start = time.perf_counter()
        a = trains_at_station(data, "Chennai", "Mon")
        scan_time = time.perf_counter() - start
        start = time.perf_counter()
        idx = build_index(data)
        index_build_time = time.perf_counter() - start
        start = time.perf_counter()
        b = query_index(idx, "Chennai", "Mon")
        indexed_time = time.perf_counter() - start
        print(n, len(a), len(b), f"scan={scan_time:.6f}s",
              f"index-build={index_build_time:.6f}s",
              f"indexed-query={indexed_time:.6f}s",
              "index-container-bytes=", sys.getsizeof(idx))
```

An unknown station returns an empty list. A train that does not run on the requested day must be excluded. For a fair experiment, report index construction time separately from query time and measure index memory more completely if required; sys.getsizeof(index) alone does not include all nested lists and tuples.

Q7 – Design a Contact-Merging Program from First Principles (10 Marks)

Stage 1 – Requirements

- Input: contact records from multiple sources, each possibly containing name, phone numbers and email addresses.
- Output: merged contact records, each containing a display name and deduplicated sets of normalized phone numbers and emails.
- Measurable requirements: normalize phone formats; handle missing fields; merge records sharing a normalized phone; preserve all distinct numbers/emails; report ambiguous records rather than silently merging on name alone.
- Identity rule: normalized phone number is the primary key. Name alone is not a reliable unique identifier.

Stage 2 – Design and pseudocode

Normalize a phone by retaining digits and removing a leading country code 91 when the resulting Indian number has 12 digits. Normalize email by stripping whitespace and lowercasing. Normalize names by stripping, collapsing whitespace and lowercasing for comparison. Group records in a dictionary keyed by normalized phone; each value contains a display name and sets of phone numbers and emails.

```
PSEUDOCODE
FOR each contact from every source
    NORMALIZE name, phone numbers and emails
    IF no usable phone number exists
        place contact in an unresolved list
    ELSE
        FOR each normalized phone number
            IF number not in contact dictionary
                create contact record with empty phone/email sets
            ADD original/normalized phone to the record's phone set
            ADD normalized emails to the record's email set
            choose a non-empty display name if needed
RETURN merged contacts and unresolved contacts
```

Stage 3 – Implementation and validation

```
import random
import time

def normalize_phone(raw):
    """Normalize common Indian phone formats to a 10-digit number."""
    digits = "".join(ch for ch in str(raw) if ch.isdigit())
    if digits.startswith("91") and len(digits) == 12:
        digits = digits[2:]
    if len(digits) == 11 and digits.startswith("0"):
        digits = digits[1:]
    return digits if len(digits) == 10 else ""

def normalize_email(raw):
    return str(raw).strip().lower() if raw and str(raw).strip() else ""

def normalize_name(raw):
    return " ".join(str(raw or "").split())

def merge_contacts(contact_lists):
    """Merge contacts by normalized phone; return merged and unresolved."""
    merged = {}
    unresolved = []
    for source in contact_lists:
        for contact in source:
            name = normalize_name(contact.get("name", ""))
            phones = contact.get("phones", [])
            emails = contact.get("emails", [])
            normalized_phones = {normalize_phone(p) for p in phones}
            normalized_phones.discard("")
            normalized_emails = {normalize_email(e) for e in emails}
            normalized_emails.discard("")
            if not normalized_phones:
```

```
                unresolved.append(contact)
                continue
            for phone in normalized_phones:
                if phone not in merged:
                    merged[phone] = {
                        "name": name,
                        "phones": set(),
                        "emails": set()
                    }
                rec = merged[phone]
                if not rec["name"] and name:
                    rec["name"] = name
                rec["phones"].add(phone)
                rec["emails"].update(normalized_emails)
    # Group phone keys that belong to the same record into display contacts.
    # The primary phone key is the identity in this simple implementation.
    return list(merged.values()), unresolved

phone1 = [
    {"name": "Dr. Ravi", "phones": ["+91 98400 12345"], "emails": ["RAVI@EXAMPLE.COM"]},
    {"name": "Meena", "phones": ["9840012346"], "emails": []},
]
phone2 = [
    {"name": "ravi kumar", "phones": ["9840012345"], "emails": ["ravi@example.com", "rk@example.com"]},
    {"name": "Unknown", "phones": [], "emails": ["no-phone@example.com"]},
]
merged, unresolved = merge_contacts([phone1, phone2])
print("Merged contacts:")
for c in merged:
    print(c["name"], sorted(c["phones"]), sorted(c["emails"]))
print("Unresolved:", unresolved)
assert len(merged) == 2
assert any("rk@example.com" in c["emails"] for c in merged)
```

Hand check: the two Ravi entries normalize to phone 9840012345 and merge their email sets; Meena remains a separate contact; the record with no phone is unresolved. A limitation is that the primary-phone-key design does not automatically unify two records connected through different phone numbers (for example, one contact has phone A and another record has both A and B unless the implementation merges groups transitively). A production implementation should merge all records linked by any shared phone, potentially using a union-find structure or canonical contact IDs.

Measure 1,000 and 100,000 contacts

```
def synthetic_contacts(n):
    contacts = []
    for i in range(n):
        number = f"{9000000000 + (i % max(1, n//2)):010d}"
        contacts.append({
            "name": f"Person {i}",
            "phones": [number],
            "emails": [f"person{i}@example.com"]
        })
    return contacts

for n in (1_000, 100_000):
    data = synthetic_contacts(n)
    start = time.perf_counter()
    result, unresolved = merge_contacts([data])
    elapsed = time.perf_counter() - start
    print(n, "merged groups:", len(result), "unresolved:", len(unresolved),
          f"time={elapsed:.6f}s")
```

The dictionary method takes O(n) average time and O(n) space for n contact records, assuming average constant-time dictionary/set operations. Comparing every pair of contacts takes $O(n^2)$ time; it can use O(1) additional space if only comparing in place, or O(n) space if storing merge groups. For 100,000 contacts, pairwise comparison is generally unsuitable. The dictionary is better because each normalized phone is used for a direct lookup.

Stage 4 – Defend the design

A dictionary keyed by normalized phone avoids comparing every pair and naturally supports deduplicating phone/email sets. Two people with the same name but different phone numbers remain separate because name is not the identity key. The main limitation is shared, recycled, missing or incorrectly entered phone numbers; unresolved contacts need manual review and a more robust production identity model.

Overall Conclusion

Lists suit ordered daily readings and timetable stops; tuples suit fixed coordinate or record values; dictionaries support key-based counting, grouping and indexing; sets support deduplication and fast set algebra; comprehensions provide concise transformations. Data-structure choice should balance access patterns, mutability, memory, audit requirements and measured performance.

Submission checklist

- Fill in student name, register number and GitHub link.
- Create repository CSA08-<RegisterNumber>-Assignment3.
- Store each question's code, input files and screenshots in separate folders.
- Execute all TODO/reference programs and capture actual output.
- Record timings at every required input size and memory values from your Python version.
- For Q1, compare naive and running-sum timings; for Q3, compare dict, Counter and list.count; for Q4, compare list and set membership; for Q5, compare loop/comprehension/filter; for Q6, compare scan and index; for Q7, measure 1,000 and 100,000 contacts.
- Add README.md, Python version, requirements.txt, run commands and at least one progressive commit per task.